

# 第15章：指针和动态内存管理

目标：

1. 掌握C语言动态内存管理的知识
2. 能够进行动态内存管理代码的编写
3. 熟悉动态内存管理的常见错误
4. 掌握柔性数组的特点

## 1. 动态内存管理

到目前为止，在我们课程中介绍的，向内存申请空间的两种方式，创建变量和数组。

代码块

```
1  int g_val = 100;
2  int main()
3  {
4      int val = 20; //在栈空间上开辟四个字节
5      char arr[10] = {0}; //在栈空间上开辟10个字节的连续空间
6      return 0;
7  }
```

这样的方式申请到内存空间，有下面的局限性：

1. 大小是固定的，不能进行灵活调整。
2. 如果是在栈区上申请的空间，出了作用域，内存空间就回收了，不能继续使用。
3. 如果是全局变量只能等程序结束才释放空间。

但是程序对于空间的需求，不仅仅是上述的情况。有时候我们希望空间的大小可以灵活的调整，内存的申请和释放更加自由。

C语言引入了动态内存开辟，让程序员自己可以申请和释放空间，自己来管理内存，就比较灵活了。

C/C++的动态内存管理是一把双刃剑，有好的一面，也有坏的一面，后期慢慢体会。

动态内存管理是在内存的堆区上申请空间的，涉及到的函数有

`malloc` , `free` , `calloc` , `realloc` , 下面我们一一介绍。

## 2. malloc

代码块

```
1 #include <stdlib.h> //需要这个头文件
2
3 void* malloc (size_t size);
```

**功能：**向内存的堆区申请一块连续可用的空间，并返回指向这块空间的起始地址。

**参数：**`size`：要分配的内存块的字节数。

**返回值：**

- 如果开辟成功，则返回这块空间的起始地址。
- 如果开辟失败（如系统内存不足），则返回一个 `NULL` 指针，因此 `malloc` 的返回值一定要做检查。
- 返回值的类型是 `void*`，所以 `malloc` 函数并不知道开辟空间的类型，具体在使用的时候使用者自己来决定。

**注意事项：**如果参数 `size` 为0，`malloc` 的行为是标准是未定义的，取决于编译器。

## 3. free

C语言提供了另外一个函数free，函数原型如下：

代码块

```
1 void free (void* ptr);
```

**功能：**释放之前通过动态内存分配函数（如 `malloc`，`calloc`，`realloc`）申请的内存空间，`malloc` 和 `free` 都声明在 `stdlib.h` 头文件中。

**参数：**

`ptr`：指向要释放的内存块的指针。

- 如果参数 `ptr` 指向的空间不是动态开辟的，那free函数的行为是未定义的。
- 如果参数 `ptr` 是NULL指针，则函数什么事都不做。

**例子：**

数组形式

```
1 #include <stdio.h>
2 #include <stdlib.h>
```

动态内存管理形式

```
1 #include <stdio.h>
2 #include <stdlib.h>
```

```

3
4  int main()
5  {
6      int num = 0;
7      //数组形式
8      scanf("%d", &num);
9      int arr[num];
10
11     int i = 0;
12     for(i = 0; i < num;
13         i++)
14     {
15         *(ptr+i) = 0;
16     }
17     return 0;
18 }

```

```

3
4  int main()
5  {
6      int num = 0;
7      scanf("%d", &num);
8      int* ptr =
9          (int*)malloc(num*sizeof(int));
10     if(NULL != ptr)//判断ptr指针是否为空
11     {
12         int i = 0;
13         for(i = 0; i < num; i++)
14         {
15             *(ptr+i) = 0;
16         }
17     }
18     else
19     {
20         printf("malloc 失败\n");
21         return 1;
22     }
23     free(ptr);//释放ptr所指向的动态内存
24     ptr = NULL;
25
26     return 0;
27 }

```

上面这个代码中，使用 `free` 函数释放动态内存，这里只是将 `ptr` 指向的堆内存还给操作系统，不能再使用。但是并不会把指针 `ptr` 置为 `NULL`，这时一定要手动将 `ptr` 置为 `NULL`，否则 `ptr` 就变成了**悬空指针**了。

 **悬空指针**：指针曾经有效，但它指向的内存已经被释放或失效了，指针里还保留着原来的地址

## 4. calloc

C语言还提供了一个函数叫 `calloc`，`calloc` 函数也用来动态内存分配。原型如下：

代码块

```
1 void* calloc (size_t num, size_t size);
```

- 函数的功能是为 `num` 个大小为 `size` 的元素开辟一块空间，并且把空间的每个字节初始化为 0。
- 与函数 `malloc` 的区别只在于 `calloc` 会在返回地址之前把申请的空间的每个字节初始化为全 0。

例子：

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *p = (int*)calloc(10, sizeof(int));
7      if(NULL != p)
8      {
9          int i = 0;
10         for(i = 0; i < 10; i++)
11         {
12             printf("%d ", *(p+i));
13         }
14     }
15     free(p);
16     p = NULL;
17     return 0;
18 }
```

输出结果：

代码块

```
1  0 0 0 0 0 0 0 0 0 0
```

所以如果我们对申请的内存空间的内容要求初始化为 0，那么可以很方便的使用 `calloc` 函数来完成任务。

## 5. realloc

- `realloc` 函数的出现让动态内存管理更加灵活。
- 有时候我们发现过去申请的空间太小了，有时候我们又会觉得申请的空间过大了，那为了合理的使用内存，我们一定会对内存的大小做灵活的调整。那 `realloc` 函数就可以做到对动态开辟内存大小的调整。

函数原型如下：

代码块

```
1 void* realloc (void* ptr, size_t size);
```

**功能：**重新调整之前分配的内存块的大小，它可以在不丢失原有数据的情况下，扩大或缩小动态分配的内存块。

**参数：**

- `ptr` 是要调整的内存空间的起始地址，如果 `ptr` 是 `NULL` 指针，`realloc` 函数的功能类似于 `malloc` 函数。
- `size` 调整之后新大小，单位是字节。

**返回值：**

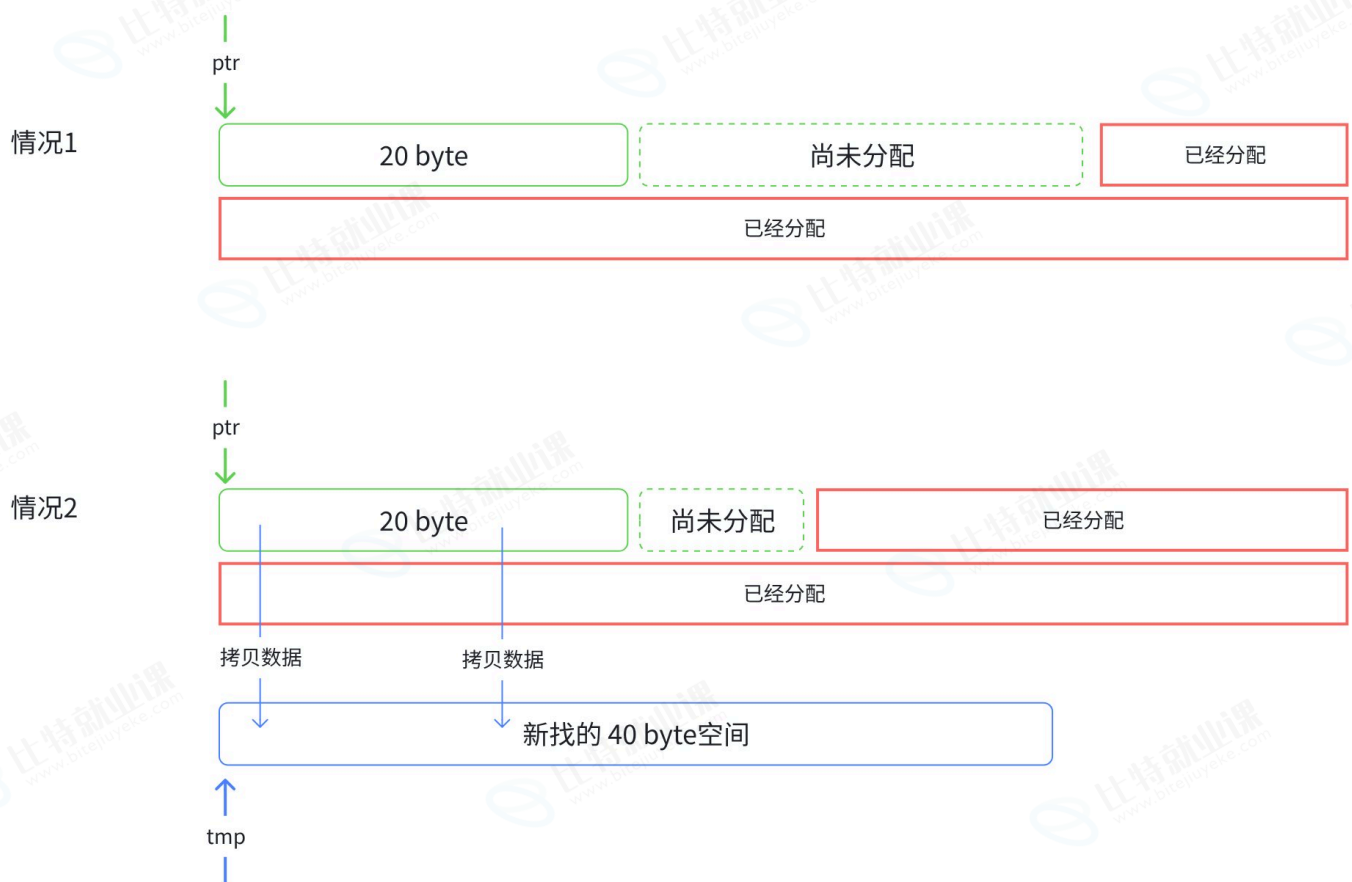
- **成功：**返回一个指向重新分配的内存块的 `void *` 类型指针。这个指针可能与原来的指针不同。
- **失败：**如果内存重新分配失败，返回 `NULL`，并且原来的内存块保持不变。

**注意事项：** `realloc` 在调整内存空间大小的时候，存在两种情况：

- **情况1：**原有空间之后有足够大的空间，要扩展的内存就直接原来内存之后直接追加空间，原来空间的数据不发生变化，最终返回的地址还是旧的地址。
- **情况2：**原有空间之后没有足够大的空间，会在内存的堆区寻找新的满足要求的空间，返回新的起始地址。在这个过程中会发生以下几件事：
  - 寻找新的满足要求的空间
  - 将旧空间的数据拷贝到新空间，保证数据不会丢失
  - 释放旧的空间，返回新空间的起始地址

代码块

```
1 int main()
2 {
3     int* ptr = (int*)malloc(20);
4     //...
5
6     //扩容空间
7     if(ptr != NULL)
8     {
9         int * tmp = (int*)realloc(ptr, 40);
10        //...
11    }
12    return 0;
```



由于上述的两种情况，`realloc` 函数的使用就要注意一些。

代码块

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *ptr = (int*)malloc(100);
7      if(ptr != NULL)
8      {
9          //业务处理
10     }
11     else
12     {
13         return 1;
14     }
15     //扩展容量
16     //代码1 - 直接将realloc的返回值放到ptr中
17     ptr = (int*)realloc(ptr, 1000); //这样可以吗? (如果申请失败会如何?)
18

```

```
19 //代码2 - 先将realloc函数的返回值放在p中, 不为NULL, 再放ptr中
20 int* p = realloc(ptr, 1000);
21 if(p != NULL)
22 {
23     ptr = p;
24     p = NULL;
25 }
26 //业务处理
27 //....
28 free(ptr);
29 ptr = NULL;
30
31 return 0;
32 }
```

## 6. 动态内存管理常见错误

### 1. 对NULL指针的解引用操作

没有对 `malloc`、`calloc`、`realloc` 函数的返回值做判断, 直接使用, 就可能导致对NULL指针的解引用操作。

代码块

```
1 void test()
2 {
3     int *p = (int *)malloc(INT_MAX/4);
4     *p = 20; //如果p的值是NULL, 就会有问题
5     free(p);
6     p = NULL;
7 }
```

### 2. 对动态开辟空间的越界访问

代码块

```
1 void test()
2 {
3     int i = 0;
4     int *p = (int *)malloc(10*sizeof(int));
5     if(NULL == p)
6     {
7         return 1;
8     }
9     for(i = 0; i <= 10; i++)
10    {
```



```
11         *(p+i) = i; //当i是10的时候越界访问
12     }
13     free(p);
14     p = NULL;
15 }
```

### 3. 对非动态开辟内存使用free释放

代码块

```
1 void test()
2 {
3     int a = 10;
4     int *p = &a;
5     free(p); //ok?
6 }
```

### 4. 使用free释放一块动态开辟内存的一部分

代码块

```
1 void test()
2 {
3     int *p = (int *)malloc(100);
4     p++;
5     free(p); //p不再指向动态内存的起始位置
6 }
```

### 5. 对同一块动态内存多次释放

代码块

```
1 void test()
2 {
3     int *p = (int *)malloc(100);
4     free(p);
5     free(p); //重复释放
6 }
```

### 6. 动态开辟内存忘记释放（内存泄漏）

代码块

```
1 void test()
```



```

2    {
3        int *p = (int *)malloc(100);
4        if(NULL != p)
5        {
6            *p = 20;
7        }
8    }
9
10   int main()
11   {
12       test();
13       while(1);
14       return 0;
15   }

```

**内存泄漏**（Memory Leak）是指在 C 语言程序中，动态分配的内存（通过 `malloc`、`calloc`、`realloc` 等函数获取）在使用完毕后，没有被正确释放（即没有调用 `free`），导致这块内存始终被程序占用而无法再被使用，直到程序运行结束。

### 为什么会发生内存泄漏？

- 忘记调用 `free`：分配内存后，后续代码路径没有释放它。
- 提前返回或异常退出：在分配与释放之间通过 `return`、`exit` 或 `goto` 跳过了释放语句。
- 丢失指针：将指向已分配内存的指针重新赋值或覆盖，导致原内存块的地址丢失，再也无法释放。

### 内存泄漏的后果：

- 资源浪费：泄漏的内存在程序生命周期内无法被回收，可用内存逐渐减少。
- 性能下降：过多内存泄漏会导致系统频繁进行内存换页、分配更大空间，程序变慢。
- 程序崩溃：当累积泄漏内存耗尽进程地址空间或系统内存时，后续 `malloc` 失败，程序可能异常终止（尤其长时间运行的服务或嵌入式系统）。
- 难以调试：内存泄漏通常在运行很久后才暴露，定位困难。

## 7. 常见笔试题解析

### 题目1：请问运行程序，会有什么样的结果？

代码块

```

1    #include <stdio.h>
2    #include <stdlib.h>
3
4    void GetMemory(char *p)
5    {

```

```
6     p = (char *)malloc(100);
7 }
8
9 void Test(void)
10 {
11     char *str = NULL;
12     GetMemory(str);
13     strcpy(str, "hello world");
14     printf(str);
15 }
16
17 int main()
18 {
19     Test();
20     return 0;
21 }
```

**题目2：请问运行程序，会有什么样的结果？**

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  char *GetMemory(void)
5  {
6      char p[] = "hello world";
7      return p;
8  }
9
10 void Test(void)
11 {
12     char *str = NULL;
13     str = GetMemory();
14     printf(str);
15 }
16
17 int main()
18 {
19     Test();
20     return 0;
21 }
```

### 题目3：请问运行程序，会有什么样的结果？

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void GetMemory(char **p, int num)
5  {
6      *p = (char *)malloc(num);
7  }
8  void Test(void)
9  {
10     char *str = NULL;
11     GetMemory(&str, 100);
12     strcpy(str, "hello");
13     printf(str);
14 }
15
16 int main()
17 {
18     Test();
19     return 0;
20 }
```

### 题目4：请问运行程序，会有什么样的结果？

代码块

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void Test(void)
5  {
6      char *str = (char *) malloc(100);
7      strcpy(str, "hello");
8      free(str);
9      if(str != NULL)
10     {
11         strcpy(str, "world");
12         printf(str);
13     }
14 }
15
16 int main()
```

```
17 {
18     Test();
19     return 0;
20 }
```

## 8. 柔性数组

也许你从来没有听说过**柔性数组（flexible array）**这个概念，但是它确实是存在的。

C99 中，结构中的最后一个元素允许是未知大小的数组，这就叫做『柔性数组』成员。

例如：

代码块

```
1 struct st_type
2 {
3     int i;
4     int a[0]; // 柔性数组成员
5 };
```

有些编译器会报错无法编译可以改成：

代码块

```
1 struct st_type
2 {
3     int i;
4     int a[]; // 柔性数组成员
5 };
```

### 8.1 柔性数组的特点

- 结构中的柔性数组成员前面必须至少一个其他成员。
- `sizeof` 返回的这种结构大小不包括柔性数组的内存。
- 包含柔性数组成员的结构用 `malloc()` 函数进行内存的动态分配，并且分配的内存应该大于结构的大小，以适应柔性数组的预期大小。

例如：

代码块

```
1 struct st_type
2 {
```

```

3     int i;
4     int a[0]; //柔性数组成员
5 };
6
7 int main()
8 {
9     printf("%d\n", sizeof(struct st_type)); //输出的是4
10    return 0;
11 }

```

## 8.2 柔性数组的使用

代码块

```

1  //代码1
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  struct st_type
6  {
7      int i;
8      int a[0]; //柔性数组成员
9  };
10
11 int main()
12 {
13     int i = 0;
14     struct st_type *p = (struct st_type*)malloc(sizeof(struct st_type) +
15 100*sizeof(int));
16     if(p == NULL)
17     {
18         perror("malloc");
19         return 1;
20     }
21     //业务处理
22     p->i = 100;
23     for(i = 0; i < 100; i++)
24     {
25         p->a[i] = i;
26     }
27     free(p);
28     p = NULL;
29
30     return 0;
31 }

```

这样柔性数组成员a，相当于获得了100个整型元素的连续空间，而且a数组的大小是可以动态调整的。

### 8.3 柔性数组的优势

上述的 `type_a` 结构也可以设计为下面的结构，也能完成同样的效果。

代码块

```
1 //代码2
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 struct st_type
6 {
7     int i;
8     int *p_a;
9 };
10
11 int main()
12 {
13     struct st_type *p = (struct st_type *)malloc(sizeof(struct st_type));
14     if(p == NULL)
15     {
16         perror("malloc");
17         return 1;
18     }
19     p->i = 100;
20     int* ptr = (int *)malloc(p->i*sizeof(int));
21     if(ptr == NULL)
22     {
23         perror("malloc 2");
24         return 2;
25     }
26     p->p_a = ptr;
27     ptr = NULL;
28
29     //业务处理
30     for(i=0; i<100; i++)
31     {
32         p->p_a[i] = i;
33     }
34
35     //释放空间
36     free(p->p_a);
37     p->p_a = NULL;
38 }
```

```
39     free(p);
40     p = NULL;
41     return 0;
42 }
```

上述 代码1 和 代码2 可以完成同样的功能，但是 方法1 的实现有两个好处：

### 第一个好处是：方便内存释放

如果我们的代码是在一个给别人用的函数中，你在里面做了二次内存分配，并把整个结构体返回给用户。用户调用free可以释放结构体，但是用户并不知道这个结构体内的成员也需要free，所以你不能指望用户来发现这个事。所以，如果我们把结构体的内存以及其成员要的内存一次性分配好了，并返回给用户一个结构体指针，用户做一次free就可以把所有的内存也给释放掉。

### 第二个好处是：这样有利于访问速度。

连续的内存有益于提高访问速度，也有益于减少内存碎片。（其实，我个人觉得也没多高了，反正你跑不了要用做偏移量的加法来寻址）

扩展阅读：

[C语言结构体里的数组和指针](#)